Performance comparison

storefront · 742d417 → e94a922 · Summary · generated 17 Oct 2026, 16:45

BEFORE
742d417

Author	Demo Developer
Date	15 Oct 2026, 10:24
Message	Product list with a live ticker
Branch	main
Scenario	perf-profiler demo
Runs	3 (page load)

AFTER
e94a922

Author	Demo Developer
Date	16 Oct 2026, 16:05
Message	Memoise list rows and move the ticker out of App
Branch	main
Scenario	perf-profiler demo
Runs	3 (page load)

Improved

Key metrics got better and none got significantly worse. Medium confidence: see the comparability notes.

✓ 5 improved ✕ 2 degraded = 19 unchanged 1 not measured

Highlights

- ✓ Wasted renders: 59 → 0 (–100%)
- ✓ JavaScript CPU time: 1.58 s → 110 ms (–93%)
- ✕ TickHeading (src/main.jsx) — renders: 0 → 60 (new)
- ✓ Row (src/Row.jsx) — renders: 7,200 → 120 (–98%)
- ✓ App (src/main.jsx) — renders: 60 → 1 (–98%)
- ✓ Wasted (src/Wasted.jsx) — renders: 60 → 1 (–98%)

Key metrics

METRIC	BEFORE	AFTER	CHANGE	Δ%	RESULT
Total blocking time key range 0 ms–0 ms → 0 ms–0 ms	0 ms	0 ms	0 ms (0%)		= Unchanged
Long tasks (> 50 ms) key range 0–0 → 0–0	0	0	0 (0%)		= Unchanged
Slowest interaction key	–	–	–		– n/a
Largest contentful paint key range 136 ms–168 ms → 116 ms–124 ms	140 ms	116 ms	–24.0 ms (–17%)		= Unchanged
Layout shift (CLS) key range 0.0015–0.0015 → 0.0015–0.0015	0.0015	0.0015	0.000 (0%)		= Unchanged
JavaScript CPU time key range 1.57 s–1.58 s → 108 ms–118 ms	1.58 s	110 ms	–1.47 s (–93%)		✓ Improved
Forced reflows key range 0–0 → 0–0	0	0	0 (0%)		= Unchanged

METRIC	BEFORE	AFTER	CHANGE	Δ%	RESULT
Wasted renders key range 59–59 → 0–0	59	0	–59 (–100%)		✓ Improved
JS heap growth key range -6.2 MB–7.9 MB → 1.2 MB–2.1 MB	2.0 MB	1.4 MB	–0.6 MB (–30%)		= Unchanged

Other metrics that moved

METRIC	BEFORE	AFTER	CHANGE	Δ%	RESULT
Scripting range 1.50 s–1.50 s → 49.4 ms–50.7 ms	1.50 s	49.8 ms	–1.45 s (–97%)		✓ Improved
Component renders range 7,380–7,380 → 183–183	7,380	183	–7,197 (–98%)		✓ Improved
React render time range 1.46 s–1.46 s → 30.5 ms–31.3 ms	1.46 s	30.6 ms	–1.43 s (–98%)		✓ Improved
DOM nodes range 405–1,241 → 1,241–2,063	405	1,652	+1,247 (+308%)		✕ Degraded
Event listeners range 261–792 → 792–1,320	261	1,056	+795 (+305%)		✕ Degraded

Components and files that changed most

WHERE	WHAT CHANGED	RESULT
Component **Row** `src/Row.jsx:3`	✓ Renders: 7,200 → 120 (–98%) Wasted renders: 0 → 0 ✓ Render time: 1.44 s → 24.6 ms (–98%)	✓ Improved
Component **App** `src/main.jsx:42`	✓ Renders: 60 → 1 (–98%) Wasted renders: 0 → 0 ✓ Render time: 8.2 ms → 0.80 ms (–90%)	✓ Improved
Component **Wasted** `src/Wasted.jsx:2`	✓ Renders: 60 → 1 (–98%) ✓ Wasted renders: 59 → 0 (–100%) Render time: 0.10 ms → 0 ms	✓ Improved
Component **Panel** `src/Panel.jsx:4`	✓ Renders: 60 → 1 (–98%) Wasted renders: 0 → 0 Render time: 0.30 ms → 0.30 ms	✓ Improved
File **src/Row.jsx**	CPU time: 3.7 ms → 0.31 ms Blocking time: 0 ms → 0 ms ✓ React render time: 1.44 s → 24.6 ms (–98%) Forced reflows: 0 → 0	✓ Improved
File **src/heavy.js**	✓ CPU time: 1.42 s → 22.9 ms (–98%) Blocking time: 0 ms → 0 ms React render time: 0 ms → 0 ms Forced reflows: 0 → 0	✓ Improved
File **src/main.jsx**	CPU time: 0.82 ms → 0.52 ms Blocking time: 0 ms → 0 ms ✓ React render time: 8.2 ms → 2.3 ms (–72%) Forced reflows: 0 → 0	✓ Improved

Problems found

New problems introduced (6)

CRITICAL · Network — react-dom-hmN8LOy-.js?… is 2.47 MB over the wire

Fix: Split the bundle, drop unused dependencies, or lazy-load it.

WARNING · Network — react-dom-hmN8LOy-.js?… is served uncompressed (2532 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Network — react.js?… is served uncompressed (217 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Memory — Large DOM: 2,063 nodes

Fix: Virtualise long lists and tables, and unmount hidden tabs/modals instead of hiding them.

… and 2 more in the detailed report.

Problems resolved (12)

CRITICAL · React — Row is slow to render: 1442ms total, 0.2ms per render (7200×)

`Row()` in `src/Row.jsx:3`

Fix: Move expensive calculations into useMemo, virtualise long lists, and avoid deriving large data structures inside render.

CRITICAL · CPU — src/heavy.js spent 1420ms of JavaScript (90% of CPU)

`burn()` in `src/heavy.js:2`

Fix: Look for work repeated on every render/event, loops over large data, or JSON/regex/sort work that can be cached, memoized or moved to a worker.

CRITICAL · Network — react-dom-hmN8LOy-.js?… is 2.47 MB over the wire

Fix: Split the bundle, drop unused dependencies, or lazy-load it.

WARNING · React — Row gets a new function for “onPick” on every render (7080×)

`Row()` in `src/Row.jsx:3`

Fix: Create “onPick” with useCallback (or hoist it out of the parent).

… and 8 more in the detailed report.

Problems that persist (1)

ISSUE	IMPACT BEFORE	IMPACT AFTER
warning · Row rendered 120× (39/s)	1.44 s	25.0 ms

Can this comparison be trusted?

4 of 6 comparability checks passed.

! **Different file-attribution modes.** sourcemap + urls / sourcemap: function lines may differ between sides.

! **Some recordings were nearly idle.** 3 run(s) executed under 300 ms of JavaScript: little was measured.

A change counts only if it exceeds the metric's noise floor and the run-to-run spread. The detailed report lists every metric, component, file and function, plus the methodology.